Software Architecture and Design Specification

Pocket Packet Sniffer

Project: Pocket Packet Sniffer — Wi-Fi / Ethernet Packet Sniffing & Analysis Dashboard

Version: 1.0

Authors: CHINMAYI HEBBAR AM, HAMZA SHABBIR SAHAPURWALA, GOWTHAM L, CHENGAPA KAIBILIRA SANTOSH

Date: 13-09-2026

Status: Draft v1.0

Revision History

Version	Date	Author	Change Summary
0.1	13-09-2026	Team Lead	Initial Software Architecture and Design Specification derived from SRS v1.0 and Test Plan v1.0: component architecture, sequence diagrams, API design, and security architecture for the Pocket Packet Sniffer.

Approvals

Role	Name	Signature / Date
Team Lead / System Architect	CHINMAYI HEBBAR A M	14-09-2026
Network Capture Engineer	CHENGAPA KAIBILIRA SANTOSH	14-09-2026
Packet Analysis Engineer	GOWTHAM L	14-09-2026
Dashboard & QA Engineer	HAMZA SHABBIR SAHAPURWALA	14-09-2026

1. Introduction

1.1 Purpose

This document specifies the software architecture and design of the Pocket Packet Sniffer (PPS) — a software-based Wi-Fi/Ethernet packet capture, Zeek-driven traffic analysis, and web-dashboard system. It translates the functional, non-functional, and security requirements defined in the Pocket Packet Sniffer SRS v1.0 into a concrete component architecture, interaction design, and interface specification to guide implementation, code review, and test-case derivation.

1.2 Scope

This document covers the architecture and design of the capture, analysis, backend/API, dashboard, and storage components of PPS: multi-interface (Wi-Fi/Ethernet) packet capture, Zeek integration and anomaly detection, REST/WebSocket backend services, local data storage and export, authentication and audit logging, and the web dashboard UI. Consistent with the SRS and Test Plan, out of scope: cloud-hosted multi-tenant deployment, dedicated/custom capture hardware, decryption of TLS-protected payloads, and long-term (> 90 day) data warehousing.

1.3 Audience

Developers, QA/test engineers, security reviewers, course instructors/evaluators, and future maintainers of the Pocket Packet Sniffer project.

1.4 Definitions

PPS (Pocket Packet Sniffer), Wi-Fi, Zeek, pcap, BPF, OUI, UI, API, TLS, RTM, REST, WebSocket, JWT (JSON Web Token), RBAC (Role-Based Access Control), STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege — threat-modeling framework).

2. Document Overview

2.1 How to use this document

This document provides the architectural deliverables for Pocket Packet Sniffer, including a component (UML) diagram, sequence diagrams for key workflows, API design, and a threat model, structured to support implementation, code review, and derivation of test cases alongside the Test Plan (STP).

2.2 Related Documents

SRS — Pocket Packet Sniffer Software Requirements Specification v1.0. STP — Pocket Packet Sniffer Software Test Plan v1.0. RTM — Requirements Traceability Matrix, SRS Section 8.

3. Architecture

3.1 Goals & Constraints

Goals: a modular, maintainable architecture with documented module interfaces (PPS-NF-005); dashboard live-statistics latency ≤ 2 seconds under normal capture load (PPS-NF-001); reliable capture sessions of at least 4 hours without crash or data loss (PPS-NF-003); cross-platform operation on Windows and Linux (PPS-NF-002); secure-by-default handling of captured data and dashboard access (PPS-SR-001..006).

Constraints: tooling fixed to Python (capture, backend, glue logic) and Zeek (traffic analysis); Zeek throughput is capped by host-machine CPU/RAM; no dedicated or custom capture hardware is used — only the host's available Wi-Fi/Ethernet interfaces; capture is restricted to networks/devices the team owns or has explicit permission to test.

3.2 Stakeholders & Concerns

- Network Administrators / Field Analysts: simple start/stop workflow, accurate live statistics, clear alerts.
- Course Evaluators / Reviewers: exportable, readable session reports.
- Security reviewer / instructor: authentication, audit logging, and protection of captured data.
- Developers / maintainers: modularity, documented interfaces, testability.

3.3 Component (UML) Diagram

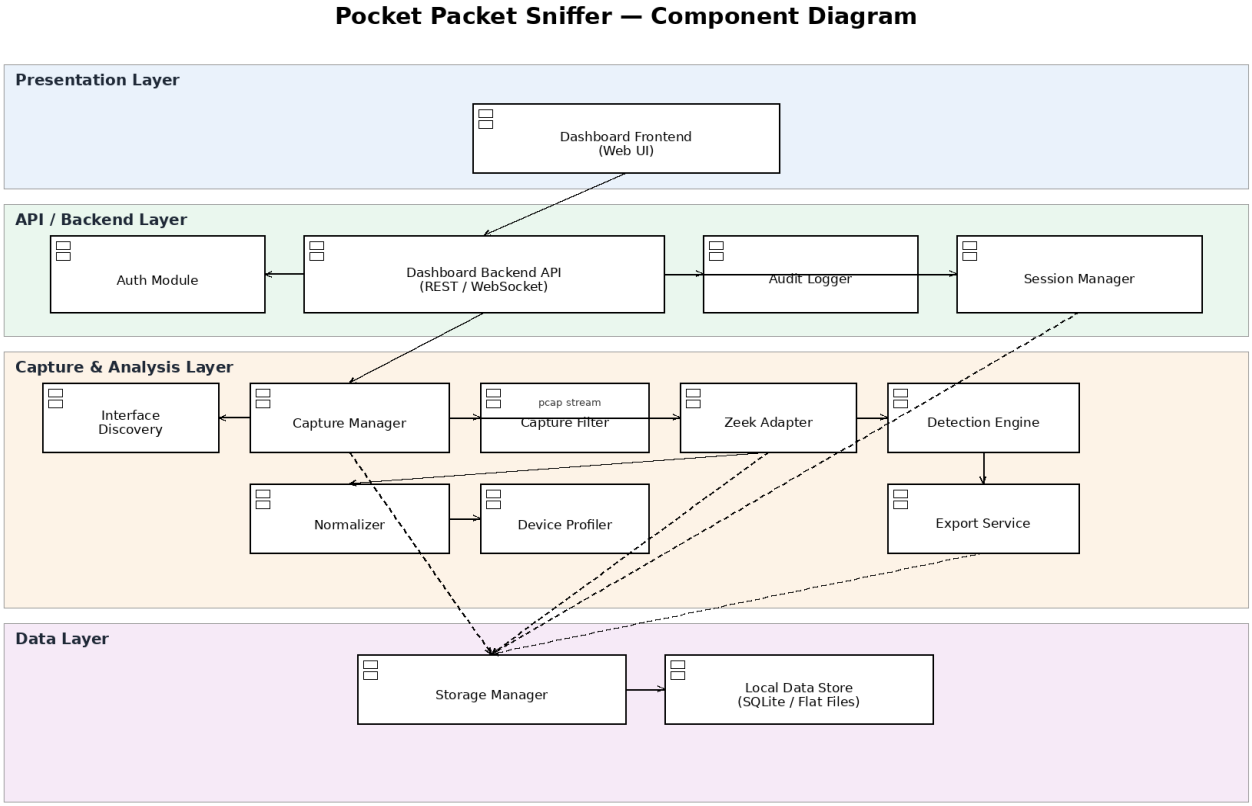


Figure 1. Pocket Packet Sniffer component diagram.

3.4 Component Descriptions

- Interface Discovery: enumerates the host's available Wi-Fi and Ethernet interfaces for selection (PPS-F-001, PPS-F-004).
- Capture Manager: starts, stops, pauses, and switches capture sessions; persists capture settings/filters (PPS-F-002..004, PPS-F-018, PPS-F-020).
- Capture Filter: applies user-defined BPF-style filters by protocol, host, or port (PPS-F-005).
- Zeek Adapter: streams captured pcap data to Zeek and reads back conn/dns/http/ssl/weird/notice logs in near real time (PPS-F-006, PPS-F-007).
- Detection Engine: evaluates Zeek events and custom detection scripts against policy rules and raises security alerts (PPS-F-008, PPS-F-014).
- Normalizer: timestamps and normalizes captured packet metadata and Zeek log records into the internal data model (PPS-F-009).
- Device Profiler: resolves MAC/OUI vendor information and builds the active-host list (PPS-F-010, PPS-F-012).
- Dashboard Backend API (REST/WebSocket): serves live statistics, device lists, and alerts to the frontend, and exposes session-control endpoints (PPS-F-011..014).
- Dashboard Frontend (Web UI): responsive browser UI for live stats, protocol distribution, device list, alerts, filters, and reporting (PPS-F-011..014, PPS-NF-004).
- Session Manager: session naming/tagging and historical session browsing (PPS-F-017).
- Storage Manager: persists pcap files and Zeek logs locally with configurable retention/rotation, up to 5 GB per session (PPS-F-015, PPS-NF-006).
- Export Service: exports session data as pcap, Zeek logs, and CSV reports (PPS-F-016).
- Auth Module: authenticates dashboard users, stores salted password hashes, and enforces first-run credential setup (PPS-F-021, PPS-SR-001, PPS-SR-004).

- Audit Logger: records administrative actions (session start/stop, configuration changes, exports, login attempts) for audit purposes (PPS-SR-006).

3.5 Chosen Architecture Pattern and Rationale

A layered, modular-monolith architecture is chosen: a Capture layer, an Analysis layer (Zeek integration and detection), a Backend/API layer, and a Presentation/Dashboard layer, each with documented interfaces (satisfying PPS-NF-005). This keeps capture, analysis, and presentation concerns cleanly separated while allowing all components to run as a single deployable Python application on one host — appropriate for a small team and single-host deployment as specified in the SRS operating environment. A distributed microservices architecture was considered and rejected: it would introduce network hops and operational complexity (service discovery, inter-service auth) disproportionate to the project's single-host, single-operator scope, and would work against the low-latency dashboard requirement (PPS-NF-001).

3.6 Technology Stack & Data Stores

- Language/runtime: Python 3.x for capture control, backend, and glue logic.
- Packet capture: Python packet-capture libraries (e.g., Scapy, PyShark) over Npcap (Windows) / libpcap (Linux).
- Analysis engine: Zeek, invoked as a subprocess/log-tailing integration from the Python backend.
- Backend/API: Flask or FastAPI, exposing REST endpoints and a WebSocket channel for live push updates.
- Data store: SQLite and/or flat files for session metadata, parsed device/alert records, and log references.
- Frontend: HTML/CSS/JavaScript dashboard, responsive for desktop and mobile browsers, consuming REST and WebSocket APIs.
- Security: HTTPS/TLS for transport where feasible; bcrypt/argon2 for password hashing.
- Source control/CI: Git/GitHub.

3.7 Risks & Mitigations

Risk	Mitigation
Zeek is unavailable or misconfigured on the host.	Verify/install/configure Zeek before enabling Zeek-dependent features; surface a clear status/error in the dashboard; run Zeek-dependent tests only in a validated environment.
Required Wi-Fi/Ethernet interface or capture permissions are unavailable.	Interface Discovery validates accessible interfaces and reports actionable errors; documentation specifies required OS-level capture permissions.
Host resource limits cap sustained Zeek analysis throughput.	Apply capture filters to reduce load, use log rotation/retention (PPS-NF-006), and validate throughput against PPS-NF-001/003 in performance and reliability testing.
Unauthorized use of the application to capture traffic without permission.	Authentication and RBAC-style access control (PPS-SR-001, PPS-F-021) plus audit logging (PPS-SR-006); documented safety note restricting use to authorized networks/devices.
Sensitive data (credentials, personal data) exposed via captured traffic or exports.	Default masking/redaction of sensitive payloads (PPS-SR-005) and access-restricted, optionally encrypted storage/exports (PPS-SR-003).

3.8 Traceability to Requirements

Requirement(s)	Component(s)
PPS-F-001..005 (capture, interface selection, filtering)	Interface Discovery, Capture Manager, Capture Filter
PPS-F-006..010 (Zeek integration, detection, normalization, device profiling)	Zeek Adapter, Detection Engine, Normalizer, Device Profiler
PPS-F-011..014 (dashboard & visualization)	Dashboard Backend API, Dashboard Frontend
PPS-F-015..017 (storage, export, session history)	Storage Manager, Export Service, Session Manager

Requirement(s)	Component(s)
PPS-F-018..020 (settings persistence, dashboard access, safe session control)	Capture Manager, Auth Module, Dashboard Backend API
PPS-F-021 / PPS-SR-001, 004 (authentication)	Auth Module
PPS-NF-001 (performance)	Dashboard Backend API, Normalizer
PPS-NF-002 (portability/compatibility)	Capture Manager, Zeek Adapter (OS-independent implementation)
PPS-NF-003 (reliability)	Capture Manager, Zeek Adapter, Storage Manager
PPS-NF-004 (usability)	Dashboard Frontend
PPS-NF-005 (maintainability)	Overall layered architecture (Section 3.5)
PPS-NF-006 (scalability/storage)	Storage Manager
PPS-SR-002 (transport security)	Dashboard Backend API (HTTPS/TLS)
PPS-SR-003 (storage access restriction)	Storage Manager
PPS-SR-005 (redaction)	Dashboard Backend API, Dashboard Frontend
PPS-SR-006 (audit logging)	Audit Logger

3.9 Security Architecture

Threat Modeling (STRIDE):

- Spoofing → username/password (or token) authentication for all dashboard and API access (PPS-SR-001); HTTPS/TLS to prevent credential interception in transit (PPS-SR-002).
- Tampering → captured pcap/log files are access-restricted to the application's local user account, with optional encryption at rest for exports (PPS-SR-003).
- Repudiation → Audit Logger records session start/stop, configuration changes, exports, and login attempts with timestamps (PPS-SR-006).
- Information Disclosure → HTTPS/TLS for dashboard-backend traffic (PPS-SR-002); default redaction/masking of sensitive captured payloads such as plaintext credentials (PPS-SR-005).
- Denial of Service → capture filters and log rotation/retention bound resource usage (PPS-NF-006); API input validation guards against malformed requests.
- Elevation of Privilege → authenticated access gates all session-control and configuration actions (PPS-F-021); first-run setup forces creation of administrator credentials and forbids hardcoded secrets (PPS-SR-004).

4. Design

4.1 Design Overview

Pocket Packet Sniffer is designed as a modular, layered application. Capture components acquire raw traffic from the host's Wi-Fi/Ethernet interfaces; the analysis layer streams that traffic to Zeek and evaluates results against detection rules; the backend/API layer normalizes state and exposes it over REST and WebSocket; and the dashboard renders live statistics, device information, and alerts in the browser. Each layer exposes a narrow, documented interface to the layer above it, so the four ownership areas in SRS Appendix A (capture, analysis, dashboard, integration/security) can be developed and tested largely independently, then integrated through the backend API.

4.2 UML Sequence Diagrams

Two sequence diagrams below cover the primary interactive flows of the application: starting a capture session through to a live security alert, and authenticating to export a session report.

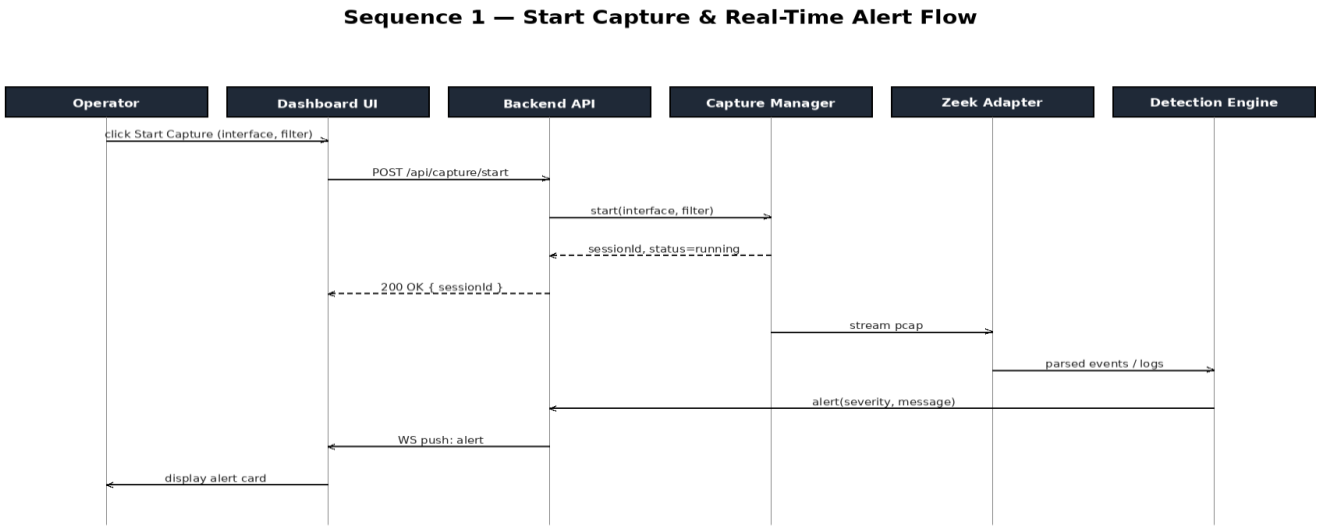


Figure 2. Sequence 1 — Start Capture & Real-Time Alert Flow (PPS-F-002, PPS-F-006..008, PPS-F-013..014).

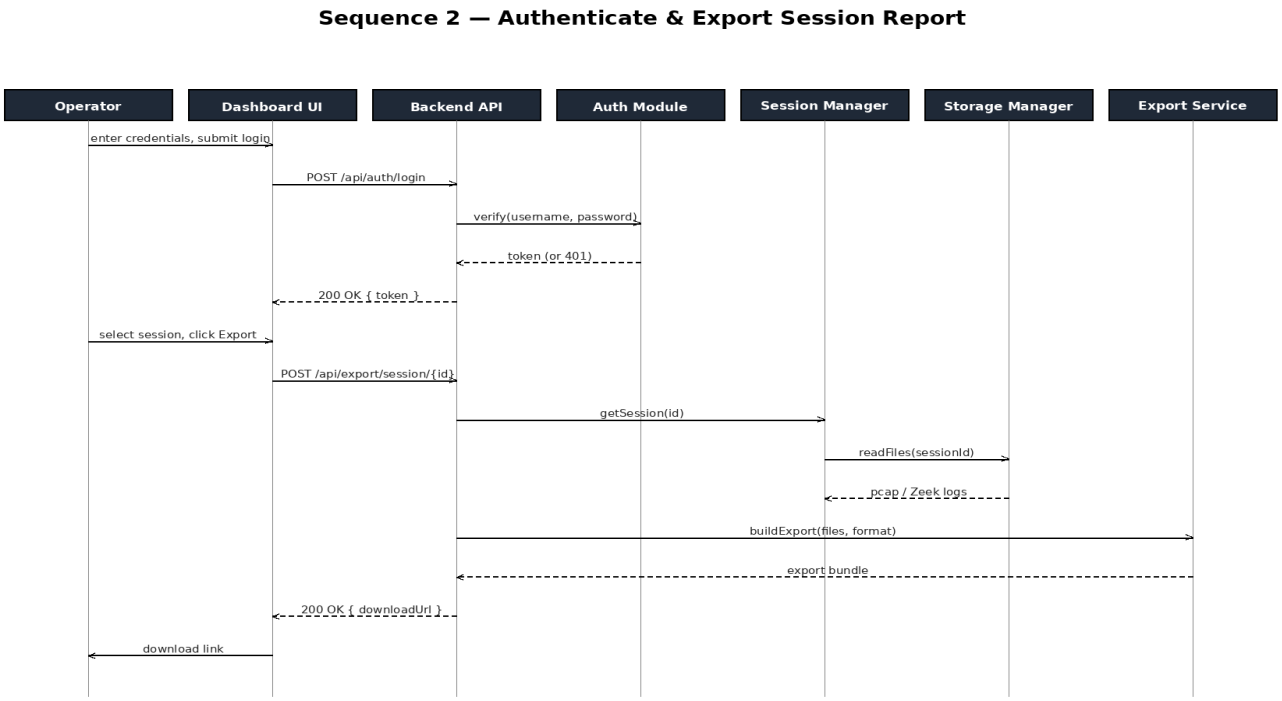


Figure 3. Sequence 2 — Authenticate & Export Session Report (PPS-F-021, PPS-F-016..017, PPS-SR-001).

4.3 API Design

Interface definitions for the Auth Module, Capture Manager, and Export Service (backend endpoints consumed by the Dashboard Frontend):

Endpoint: /api/auth/login

Method: POST

Request: { username, password }

Response: { token, expiresAt }

Errors: 401 Invalid credentials, 403 Account locked

Endpoint: /api/capture/start

Method: POST

Request: { interface, filter, sessionName } (Authorization: Bearer <token>)

Response: { sessionId, status, startedAt }

Errors: 401 Unauthorized, 400 Invalid interface/filter, 409 Capture already running

Endpoint: /api/export/session/{id}

Method: POST

Request: { format: "pcap" | "zeek-logs" | "csv" } (Authorization: Bearer <token>)

Response: { downloadUrl, expiresAt }

Errors: 401 Unauthorized, 404 Session not found, 410 Session data expired/purged

WebSocket channel: /ws/live

Pushes live traffic-statistic updates, newly discovered devices, and Zeek-flagged alerts to connected dashboard clients as they occur, avoiding client-side polling (supports PPS-F-011..014 and the PPS-NF-001 latency target).

4.4 Error Handling, Logging & Monitoring

All API errors return a standardized JSON envelope: { errorCode, message }. Application logs record operational events (capture start/stop, Zeek process health, export completion) but never sensitive content such as plaintext credentials or full packet payloads; the Audit Logger is a separate, append-only log dedicated to administrative actions for accountability (PPS-SR-006). Monitoring covers: capture-session uptime, Zeek process health, dashboard end-to-end latency (PPS-NF-001), storage utilization against the 5 GB/session target (PPS-NF-006), and alert-generation throughput.

4.5 UX Design

The dashboard presents live packet/sec and protocol-breakdown statistics, a searchable device list (IP, MAC, resolved vendor, protocol), a severity-color-coded alert panel, and session controls (start/stop/pause, interface selection, filter entry) — all reachable without command-line interaction, on both desktop and mobile browser viewports (PPS-NF-004). A dedicated reports view supports session tagging/history browsing and one-click export to pcap, Zeek logs, or CSV.

4.6 Open Issues & Next Steps

- Multi-user role-based accounts (separate Administrator and read-only Analyst roles) beyond the current single authenticated-user model.
- Deeper protocol decoders and full-text/packet-level search across stored sessions.
- Optional notification integrations (e.g., email/webhook) for critical-severity alerts.
- PDF-formatted session reports in addition to CSV/pcap/Zeek-log export.
- Containerized packaging to simplify Windows/Linux installation while preserving PPS-NF-002 compatibility.

5. Appendices

5.1 Glossary

PPS, Wi-Fi, Zeek, pcap, BPF, OUI, TLS, RTM, REST, WebSocket, JWT, RBAC, STRIDE.

5.2 References

Pocket Packet Sniffer SRS v1.0; Pocket Packet Sniffer Software Test Plan (STP) v1.0; IEEE 42010 (Recommended Practice for Architecture Description); OWASP Top 10; NIST SP 800-115 (Technical Guide to Information Security Testing and Assessment).

5.3 Tools

PlantUML / draw.io (architecture and sequence diagrams), Postman / Swagger (API design and testing), Wireshark (capture validation), Git/GitHub and Jira (source control and delivery tracking, per SRS Appendix A).